Aruco Marker Visual Relocalization Navigation

Aruco Marker Visual Relocalization Navigation

1. Course Content
   - Learning Objectives
2. Preparation
3. Recording the ArUco Tag Map Coordinates
   - 3.1 Preparing the ArUco Tags
   - 3.2 Deploying the ArUco Tags
   - 3.3 Starting the Tag Recording Node
4. Starting the Visual Relocalization Navigation Mode
   - 4.1 Verifying the Relocalization Feature
   - 4.2 Controlling Tag Detection via Service
5. Source Code Analysis
   - 5.1 Tag Recording Node Launch File
   - 5.2 Relocalization Mode Navigation Launch File
   - 5.3 aritag_relocation.yaml File Structure
   - 5.4 Visual Relocalization Working Principle
   - 5.5 Tag Configuration Parameters
   - 5.6 Inter-Node Communication Graph

1. Course Content

- Use ArUco tags combined with the pose map's constraint solving function to implement the visual relocalization navigation feature.
- By deploying ArUco tags as spatial anchors, when the robot's localization is lost or the accumulated error is too large, visual detection is used to automatically trigger relocalization, correct the robot's global pose in the map, and improve the robustness and reliability of the navigation system.

> **Note**
>
> The visual relocalization scheme here is used in subsequent OpenClaw courses and sandbox map courses to enhance global localization stability

Learning Objectives

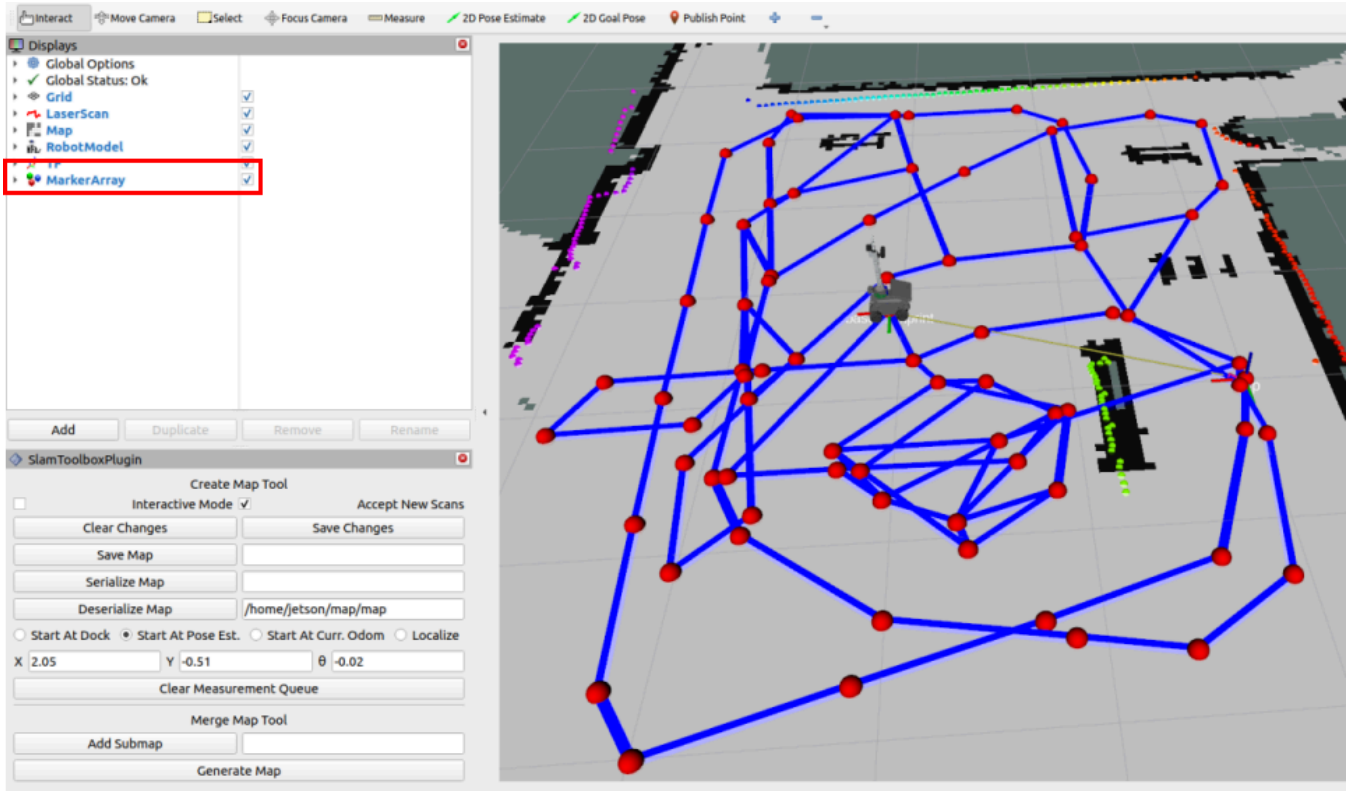
- Master the methods for preparing, deploying, and recording the positions of ArUco tags
- Understand the working principles and applicable scenarios of visual relocalization
- Learn to use the navigation mode that combines pose constraints with visual relocalization

2. Preparation

- Referring to the prerequisite course [slam-toolbox Algorithm Mapping], complete a grid map and a pose map in posegraph format; mapping and saving the map can be done either through the UI panel or CLI commands

> **Important**

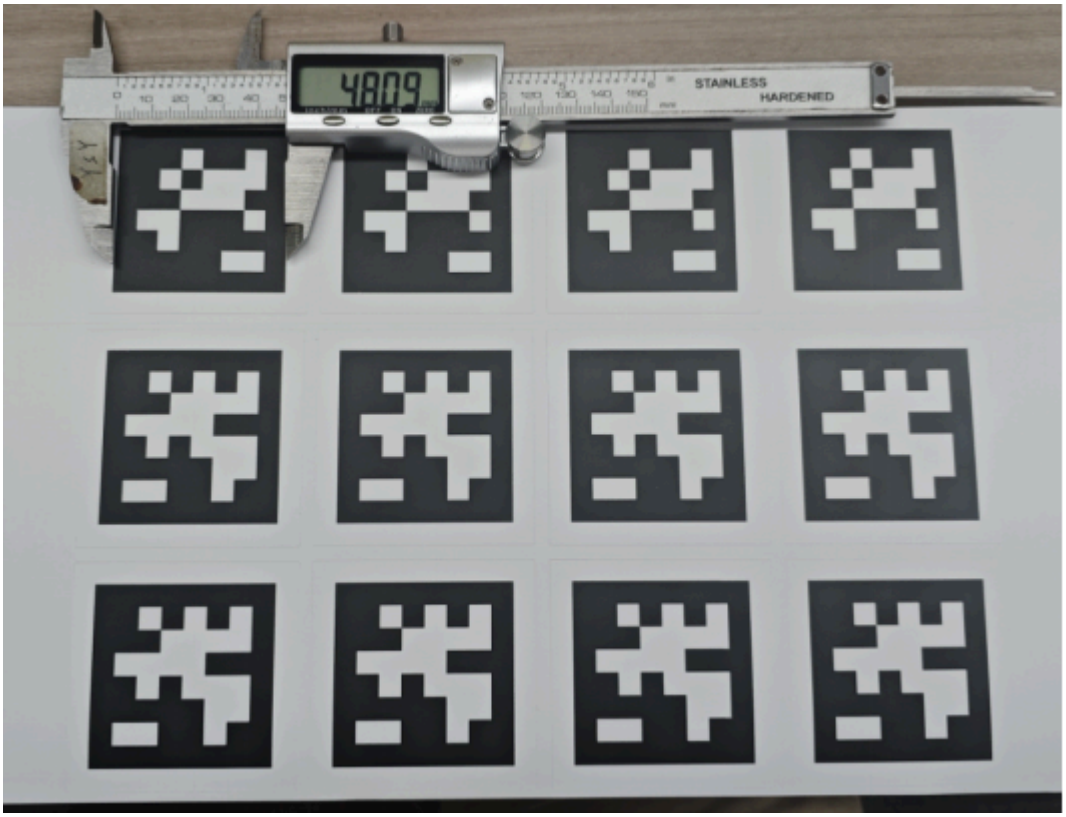
> **Pose points must be checked**: when mapping with slam_toolbox, click the MarkerArray option in the Display panel in Rviz to view the pose map's pose points. Make sure there are enough pose points within the map area for localization during subsequent navigation.



3. Recording the ArUco Tag Map Coordinates

3.1 Preparing the ArUco Tags

- They can be obtained in the following ways:
    1. The tags from the sandbox map package
    2. Download and print the tags yourself from the attached materials
- Determine the size of the tags
    - Measure the tags' actual side length with a measuring tool; the side length of the tags included in the sandbox map package is 0.048m
    - If the tags are printed by yourself, their actual size needs to be measured manually due to printer size scaling



- Modify the ArUco detection configuration file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

- Modify size to the actual size. If you are using the tags included with the sandbox map, the factory default size is 0.048 and does not need to be modified

```
...
apriltag:                   # node name
  ros__parameters:
    image_transport: raw  # image format: "raw" or "compressed"
    family: 36h11         # tag family name: 16h5, 25h9, 36h11
    size: 0.048             # default tag edge size in meter
...
```

3.2 Deploying the ArUco Tags

- Place the ArUco tags at the required positions in the map. They can be placed anywhere the camera can observe, such as on the floor or walls.

> **Important**
>
> **Placement tips**
>
> - It is recommended to place them near frequently navigated start or end points within the camera's field of view, triggering relocalization before the car departs or after it stops upon reaching the goal point
> - It is not recommended to place them along the driving path, because when the car is moving, **the**

> **robotic arm produces mechanical swaying**, causing the camera image to shake frequently and making the computed car position inaccurate

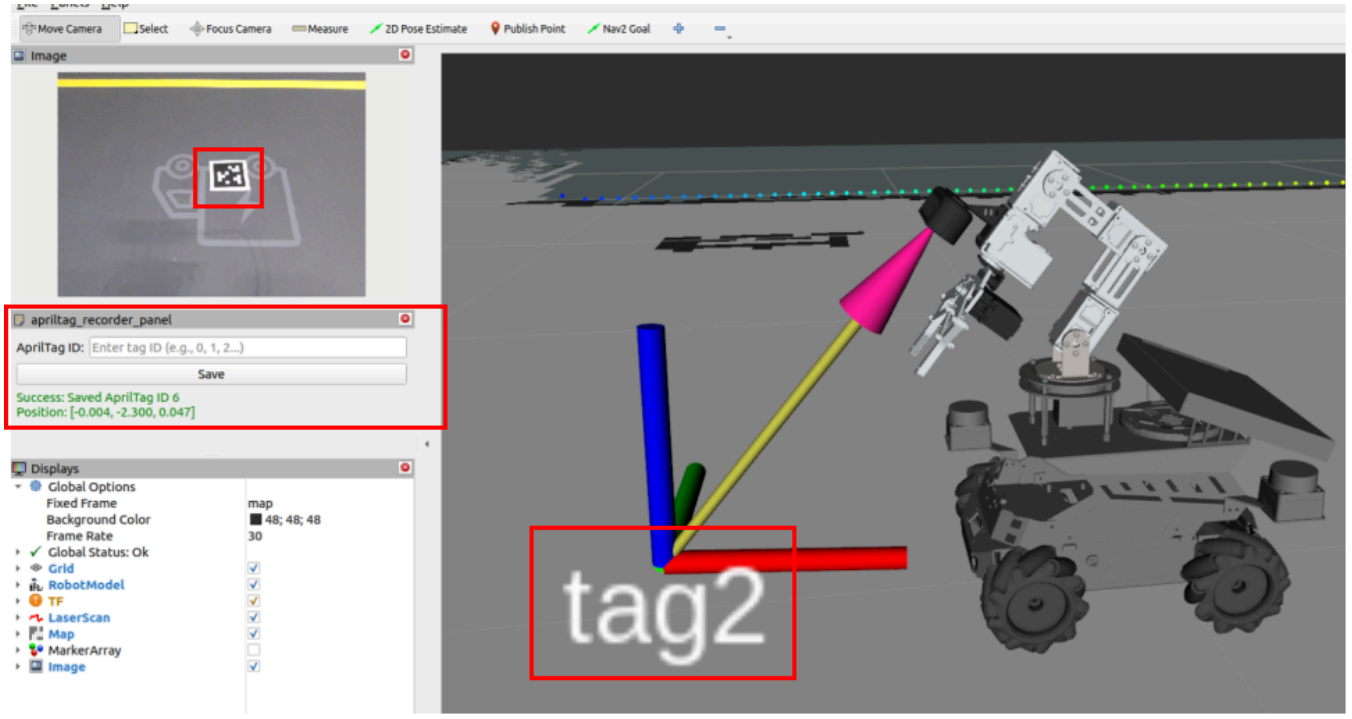


3.3 Starting the Tag Recording Node

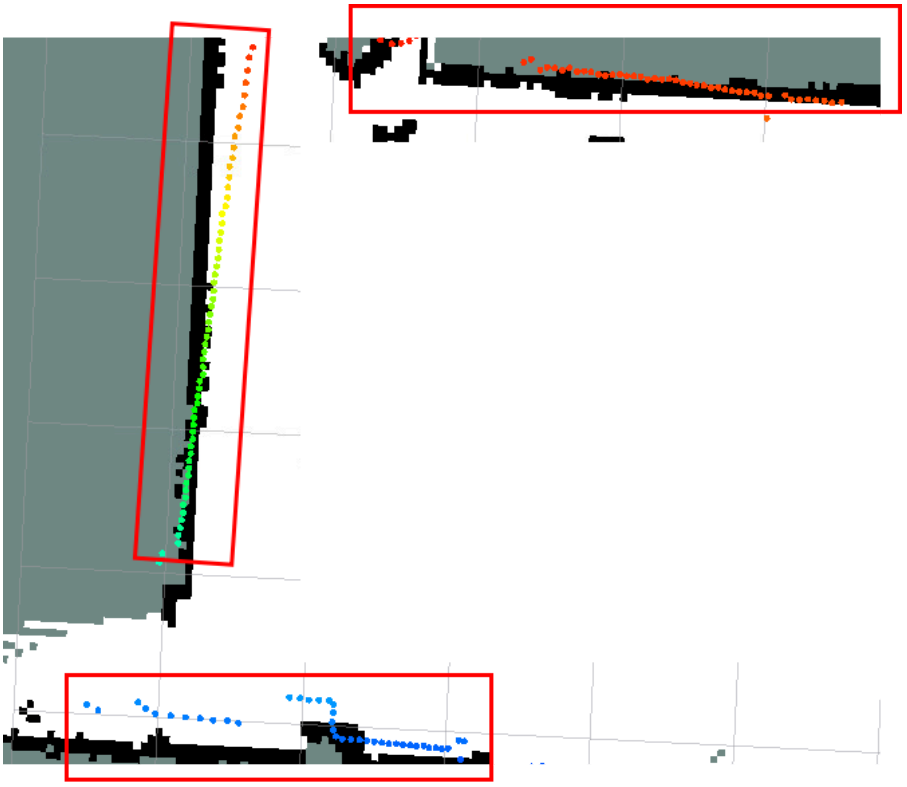
```
ros2 launch yahboomcar_bringup record_tag.launch.py
```

- Step 1: Move the car (by carrying it directly or controlling it to move) near the tag to be recorded, make the tag appear in the camera's field of view, and observe that the tag number in RVIZ looks like `Tagx`

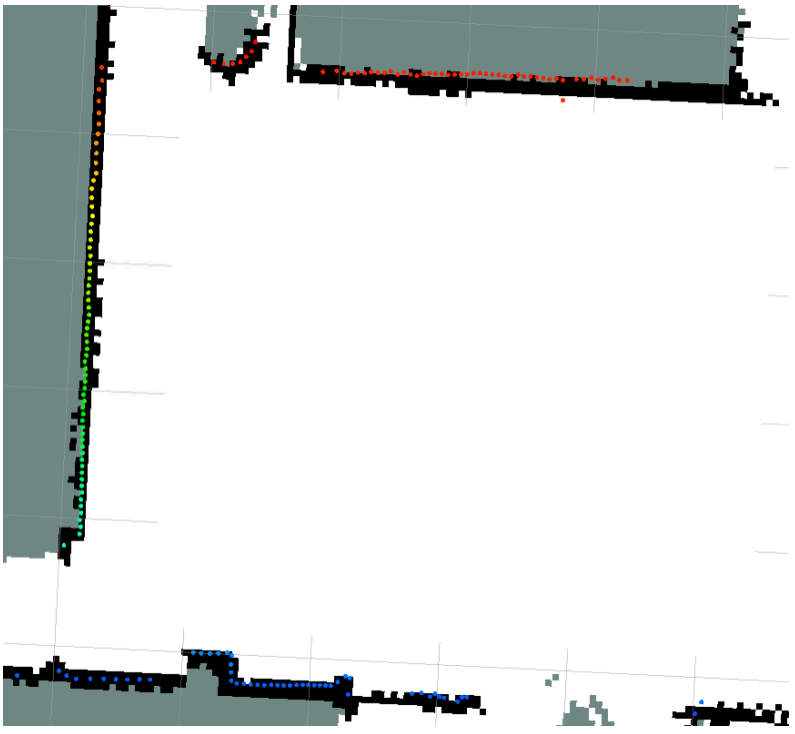
- Step 2: In the **ArUco ID** field of the **aritag_recorder_panel** panel, enter the ID of the tag to be recorded, then click **Save** to complete the recording (check whether the car's localization is accurate before recording)



- Before recording the tag position, if you find that the laser contour deviates significantly from the environmental edge features, as shown below, it proves that there is an error in the localization. In this case, you need to first use the **2D Pose Estimate** tool for manual relocalization before recording the position



- The figure below proves that the localization is accurate, with a high degree of matching between the laser contour and the environmental edge features



- Repeat the above steps to complete the position recording of all tags. Record file path:

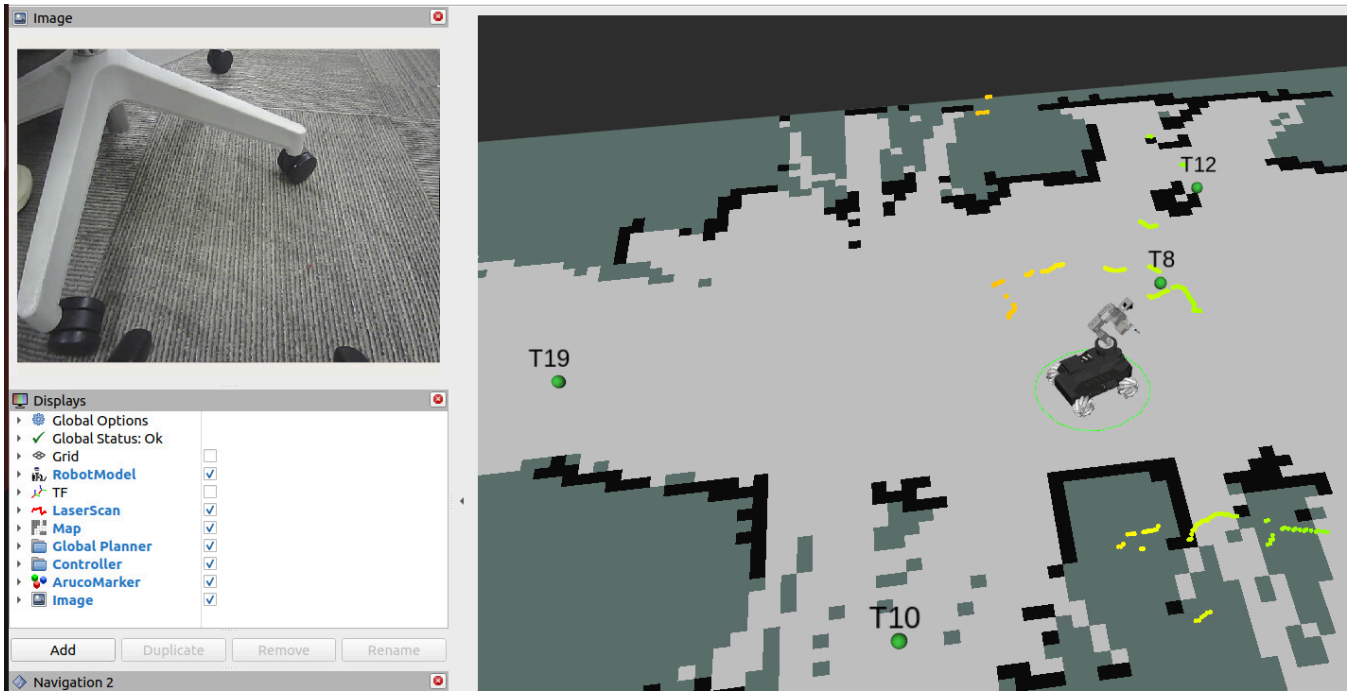
```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/aritag_relocation.yaml
```

```
jetson@yahboom:~$ cat ~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/aritag_relocation.yaml
1:
  position:
    x: 1.3833717985981973
    y: -2.9026189336934531
    z: 0.070413284960177638
  orientation:
    x: -0.031926243949601252
    y: 0.027068465461099932
    z: 0.68934845181831572
    w: 0.72321969352371984
  frame_id: map
```

4. Starting the Visual Relocalization Navigation Mode

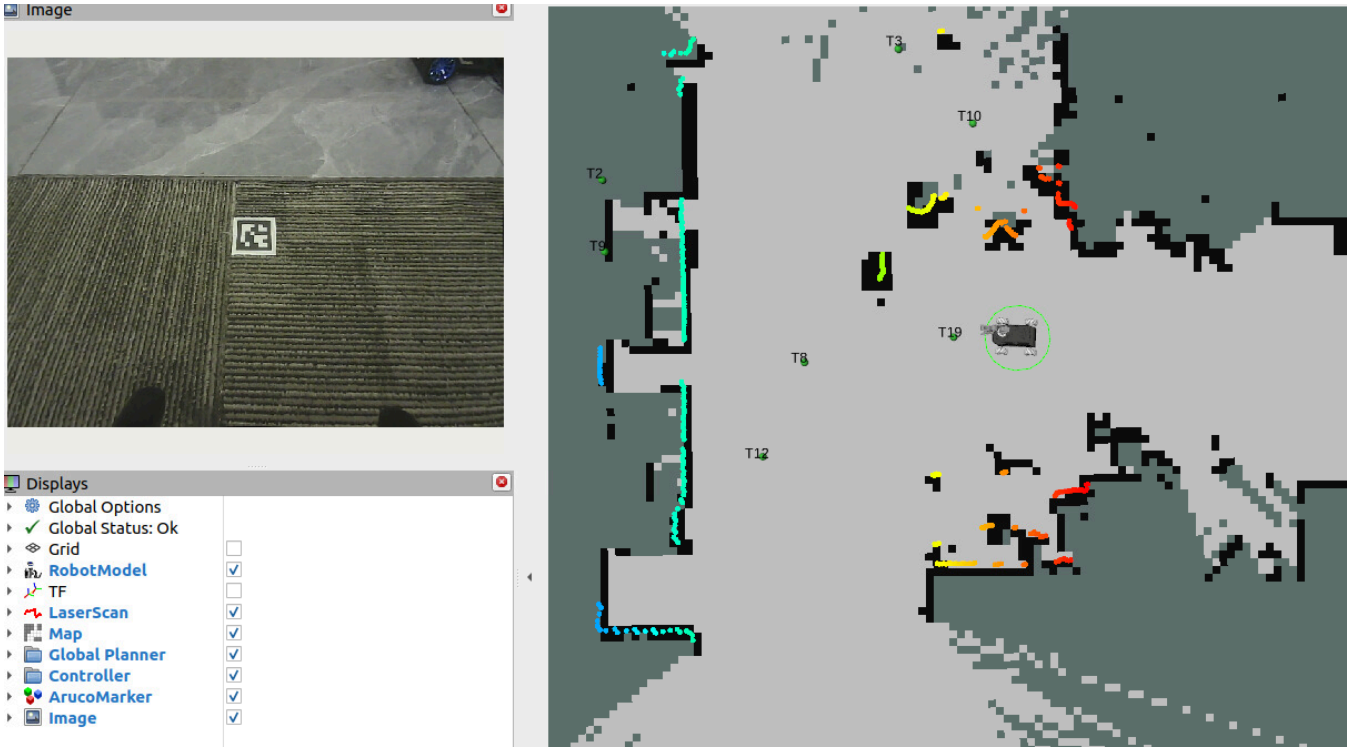
```
ros2 launch yahboomcar_bringup aruco_relocation_nav.launch.py
```

- The green circles **Txx** displayed on the map in the image are the positions of all the tags recorded in the previous steps. They may not be displayed immediately right after startup; the tag data is periodically published on the /apriltag_map_markers topic and displayed by RVIZ after subscription



4.1 Verifying the Relocalization Feature

- You can manually move the robot's position (to simulate a situation where the odometry has severe error and the global localization is chaotic, as shown in the figure above)
- When a tag in the map is seen in the camera's field of view, relocalization is automatically triggered to correct the robot's global localization in the map



- The terminal will indicate the detected Aruco marker ID through logs

```
[apriltag_relocalization-15] [INFO] [1787898223.167644213] [apriltag_relocalization]: Published 22 map markers.
[apriltag_relocalization-15] [INFO] [1787898343.165698831] [apriltag_relocalization]: Published 22 map markers.
[apriltag_relocalization-15] [INFO] [1787898348.526835350] [apriltag_relocalization]: Detected map tag 19 — attempting relocalization...
[apriltag_relocalization-15] [INFO] [1787898348.537519551] [apriltag_relocalization]: Tag 19: published /initialpose  pos=[-0.632, 0.092, 0.135]  (dist=0.62m)
```

4.2 Controlling Tag Detection via Service

- Because the aritag tag detection node occupies the CPU at a high frequency, you can turn off the detection state when detection is not needed. The node provides a control_detection service to control whether tag detection is performed
- Query status

```
ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command: 0}"
```

```
root@raspberrypi:~# ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command: 0}"
requester: making request: apriltag_localization.srv.ControlDetection_Request(command=0)

response:
apriltag_localization.srv.ControlDetection_Response(success=True, detection_enabled=True, message='Detection is currently enabled')
```

- Enable detection

```
ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command: 1}"
```

```
root@raspberrypi:~# ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command: 1}"
waiting for service to become available...
requester: making request: apriltag_localization.srv.ControlDetection_Request(command=1)

response:
apriltag_localization.srv.ControlDetection_Response(success=True, detection_enabled=True, message='Detection is running')
```

- Disable detection (reduces CPU usage)

```
ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command:
2}"
```

```
root@raspberrypi:~# ros2 service call /control_detection apriltag_localization/srv/ControlDetection "{command: 2}"
requester: making request: apriltag_localization.srv.ControlDetection_Request(command=2)

response:
apriltag_localization.srv.ControlDetection_Response(success=True, detection_enabled=False, message='Detection disabled
successfully')
```

5. Source Code Analysis

5.1 Tag Recording Node Launch File

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/record_tag.launch.py
```

- `apriltag_detect`: starts the `apriltag_node` tag detection node, loads the `common.yaml` configuration and enables detection, and remaps the camera image and camera_info topics
- `apriltag_recorder_node`: starts the `apriltag_recorder` recording node, which listens to the detected tag poses and saves them to the YAML file specified by `output_file_path`, with the coordinate frame being `target_frame` (default `map`)
- `navigation_launch`: includes `nav2.launch.py`, uses the slam_toolbox localization mode, does not start the chassis (`start_carbase: false`), and loads the dedicated `record_tag` Rviz configuration
- `call_detection_service`: calls the `/control_detection` service after a 5-second delay to enable tag detection (`command: 1`), waiting for the system to fully start before enabling to avoid resource conflicts
- `car_base_launch`: includes `car_base.launch.py`, enabling the IMU, LiDAR, and camera, and using the EKF odometry as the TF source

```
    apriltag_detect = Node(
        package='apriltag_localization',
        executable='apriltag_node',
        name='apriltag',
        parameters=
[os.path.join(get_package_share_directory('yahboomcar_bringup'),'param','common.yaml'),
                    {"detection_enabled": True}],
        remappings=[('/image_rect', GetCameraRgbTopic()),
                    ('/image_rect/camera_info', GetCameraInfo())],
        output='screen'
        )

    # AprilTag recorder node
    apriltag_recorder_node = Node(
        package='apriltag_localization',
        executable='apriltag_recorder',
```

```
        name='apriltag_recorder',
        output='screen',
        parameters=[{
            'output_file_path': LaunchConfiguration('output_file_path'),
            'target_frame': LaunchConfiguration('target_frame'),
        }]
    )
    #slam_toolbox_navigation
    navigation_launch = IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('yahboomcar_bringup'
),'launch','nav2.launch.py')),
            launch_arguments={'log_level': 'warn',
                              'start_carbase': 'false',
                              'localization_mode':'slam_toolbox',
                              'rviz_config': 'record_tag'
                              }.items())

    # Enable aritag detection
    call_detection_service = TimerAction(
        period=5.0,
        actions=[
            ExecuteProcess(
                cmd=['ros2', 'service', 'call', '/control_detection',
                     'apriltag_localization/srv/ControlDetection',
                     '{command: 1}'],
                output='screen'
            )
        ]
    )
    car_base_launch = IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('yahboomcar_bringup'
),'launch','car_base.launch.py')),
            launch_arguments={
                              'use_imu': 'True',
                              'use_lidar': 'True',
                              'use_camera': 'True',
                              'odom_tf_source': 'ekf'
                              }.items())
```

5.2 Relocalization Mode Navigation Launch File

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/aruco_relocation_nav.launch
.py
```

- `declared_arguments`: declares the `rviz` launch parameter, default value `True`, controlling whether to automatically start Rviz2
- `apriltag_relocalization`: includes `apriltag_relocalization.launch.py`, enabling tag detection

(`detection_enabled: true`), which detects tags and triggers relocalization

- `car_base_launch`: includes `car_base.launch.py`, enabling the IMU, LiDAR, and camera, and using the EKF odometry as the TF source
- `nav2`: includes `nav2.launch.py`, uses the slam_toolbox localization mode, does not start the chassis (`start_carbase: False`), and loads the `aruco_relocation_nav.rviz` configuration
- `LaunchDescription`: aggregates and returns the three modules of chassis launch, navigation, and tag relocalization

```
def generate_launch_description():
    bringup_dir = get_package_share_directory('yahboomcar_bringup')

    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "rviz",
            default_value='True',
            description="if rviz is used, set to True. If False, rviz will not be
launched.",
        )
    )
    apriltag_relocalization=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','apriltag_relocalization.la
unch.py')),
            launch_arguments={'detection_enabled': 'true',
                              'log_level': 'info'}.items())
    # Car base with camera and lidar
    car_base_launch = IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','car_base.launch.py')),
            launch_arguments={
                              'use_imu': 'True',
                              'use_lidar': 'True',
                              'use_camera': 'True',
                              'odom_tf_source': 'ekf'
                              }.items())
    nav2=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','nav2.launch.py')),
        launch_arguments={'localization_mode':'slam_toolbox',
                          'start_carbase': 'False',
                          'log_level':'warn',
                          'rviz': LaunchConfiguration('rviz'),
                          'rviz_config': 'aruco_relocation_nav.rviz'
                          }.items())

    return LaunchDescription([
        *declared_arguments,
        car_base_launch,
        nav2,
        apriltag_relocalization,
    ])
```

5.3 aritag_relocation.yaml File Structure

The structure of the `aritag_relocation.yaml` file generated after recording is complete is as follows:

```yaml
tags:
  - id: 0                    # ArUco tag ID
    position:                # Position coordinates (relative to the map coordinate frame)
      x: 1.234               # X coordinate (meters)
      y: 0.567               # Y coordinate (meters)
      z: 0.0                 # Z coordinate (meters; usually 0, representing ground level)
    orientation:             # Orientation (quaternion)
      x: 0.0
      y: 0.0
      z: 0.123
      w: 0.987
  - id: 1
    position:
      x: 2.345
      y: 1.234
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.456
      w: 0.890
```

Field description:

Field	Type	Description
`id`	int	The unique identifier ID of the ArUco tag
`position.x`	float	The tag's X coordinate in the map coordinate frame (meters)
`position.y`	float	The tag's Y coordinate in the map coordinate frame (meters)
`position.z`	float	The tag's Z coordinate in the map coordinate frame (meters)
`orientation.x/y/z/w`	float	The tag's orientation quaternion, representing the tag's facing direction

5.4 Visual Relocalization Working Principle

1. **Detect the ArUco tags**: the camera continuously detects ArUco tags within its field of view
2. **Match global coordinates**: if a tag is detected, read its global coordinates from `aritag_relocation.yaml`
3. **Calculate the deviation**: compare the position estimated by the current odometry with the true position provided by the ArUco tag
4. **Correct the pose**: use the ArUco tag's position to correct the robot's global pose

Relocalization advantages:

- ✅ **Eliminates accumulated error**: the odometry accumulates error over time, while the ArUco tag provides an absolute position reference
- ✅ **Solves degraded scenarios**: in areas with similar features such as long corridors, laser SLAM may produce chaotic localization, while the ArUco tag provides a reliable anchor

Notes:
⚠️ Before recording the tag position, you must ensure that the laser localization is accurate (the laser contour matches the environmental edges)
⚠️ If a tag's position changes, its global coordinates need to be recorded again

5.5 Tag Configuration Parameters

- Configuration parameter file path for the tag node

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter description:

Parameter	Description	Default Value
`image_transport`	Image transport format; `raw` is raw image, `compressed` is compressed image	`raw`
`family`	ArUco tag family type; different families use different encoding methods	`36h11`
`size`	The tag's physical side length (unit: meters); must match the actual size	`0.048`
`profile`	Whether to output performance profiling information	`false`
`max_hamming`	Maximum allowed Hamming distance (number of correctable bits); 0 means no error correction	`0`
`detector.threads`	Number of detection threads	`1`
`detector.decimate`	Pyramid downsampling factor; a larger value makes detection faster but reduces accuracy	`2.0`
`detector.blur`	Gaussian blur sigma value, used for quad detection	`0.0`
`detector.refine`	Whether to enable edge sharpening for precise localization	`true`
`detector.sharpening`	Sharpening strength of the decoded image	`0.25`
`detector.debug`	Whether to output debug images to the current working directory	`false`

Parameter	Description	Default Value
`pose_estimation_method`	Pose estimation method; `pnp` means using the PnP algorithm	`pnp`
`tag.ids`	List of tag IDs to detect; only tags appearing in the list are detected	`[1-20]`
`tag.frames`	List of TF coordinate frame names corresponding to the IDs	`[tag1-tag20]`

Important parameter tuning suggestions:

- `size`: must be measured accurately and set; errors will cause incorrect pose computation
- `family`: must match the tag family used, otherwise detection will fail
- `tag.ids`: only list the tag IDs actually used, to reduce unnecessary TF publishing

```
...
apriltag:                 # node name
  ros__parameters:
    image_transport: raw  # image format: "raw" or "compressed"
    family: 36h11         # tag family name: 16h5, 25h9, 36h11
    size: 0.048             # default tag edge size in meter
    profile: false        # print profiling information to stdout
    # tuning of detection (defaults)
    max_hamming: 0        # maximum allowed hamming distance (corrected bits)
    detector:
      threads: 1          # number of threads
      decimate: 2.0       # decimate resolution for quad detection
      blur: 0.0           # sigma of Gaussian blur for quad detection
      refine: true        # snap to strong gradients
      sharpening: 0.25    # sharpening of decoded images
      debug: false            # write additional debugging images to current working
directory
    pose_estimation_method: "pnp" # method for estimating the tag pose
    tag:
      ids:    [1, 2, 3, 4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]         # tag IDs for
which to publish transform
      frames: [tag1, tag2, tag3, tag4, tag5, tag6, tag7, tag8, tag9, tag10, tag11, tag12,
tag13, tag14, tag15, tag16, tag17, tag18, tag19, tag20]
...
```

5.6 Inter-Node Communication Graph

- Start it while the relocalization navigation is running:

```
ros2 run rqt_graph rqt_graph
```

- The apriltag node subscribes to the camera topic to detect tags and publishes the /detections topic; the relocalization node apriltag_relocation subscribes to /detections. When a tag is detected, based on the tag's recorded map anchor coordinates, it computes the position of the car's base_link in the map coordinate frame and publishes it to /initialpose, triggering the localization function of the slam_toolbox node to compute the precise coordinate position of the car in the map from the pose constraint graph

